# CONCEPTION DE “e-tasiko”

# Chapitre 4. Analyse préalable

## 4.1. Analyse de l’existant

### *4.1.1. Organisation actuelle (traitement actuel et personnel impliqué)*

- **Traitement actuel :** La gestion des tâches s'effectue de manière informelle et fragmentée. Les équipes utilisent des listes de tâches sur papier, des fichiers Excel partagés mais statiques, ou des fils de discussion (e-mails, messageries instantanées). Les informations s'égarent fréquemment, les priorités de la journée sont floues, le calcul du temps passé sur un livrable est estimé à la louche et le suivi de l'avancement global reste purement manuel.
- **Personnel impliqué :** Le Chef de projet (qui attribue et tente de suivre), les collaborateurs (qui exécutent et reportent), et les clients (en attente de visibilité).

### *4.1.2. Inventaire des moyens matériels et logiciels*

- **Matériels :** Ordinateurs de bureau, ordinateurs portables, serveurs locaux ou infrastructures Cloud.
- **Logiciels :** Navigateurs web modernes, tableurs (Microsoft Excel, Google Sheets), outils de messagerie, serveurs de bases de données rudimentaires.

## 4.2. Critique de l’existant (points forts et points faibles)

- **Points forts :** Flexibilité totale immédiate, coût financier direct nul, aucune formation logicielle requise pour les équipes.
- **Points faibles :** Perte critique d'informations, absence totale de visibilité en temps réel sur la charge de travail, rapports collaboratifs complexes à distance, oublis fréquents des dates limites par manque de rappels automatiques, et incapacité de chronométrer précisément les tâches.

## 4.3. Conception avant-projet

### *4.3.1. Proposition des solutions*

- **Solution 1 (Manuelle améliorée) :** Déploiement de tableaux physiques de type Kanban (tableaux blancs, post-it) dans les bureaux. *Solution rejetée* car inadaptée au travail à distance et au suivi automatique du temps.
- **Solution 2 (Numérique centralisée et découplée) :** Développement d'une application web dynamique et moderne baptisée **e-tasiko**. Elle centralise les données, intègre un système de suivi du temps (Time Tracking) visuel, gère les documents joints et offre une interface fluide accessible en temps réel. **(Solution retenue)**.

### *4.3.2. Méthodes de conception et outils utilisés*

- **Méthode de conception : Agile Scrum**. Elle garantit des livraisons incrémentales à chaque fin de Sprint, une adaptation continue face aux retours visuels sur l'interface et une transparence totale.
- **Outils et technologies choisis :**
    - **Frontend : Angular**. Framework robuste permettant de bâtir une Single Page Application (SPA) dynamique, offrant une réactivité parfaite pour les compteurs de temps et les fenêtres modales d'**e-tasiko**.
    - **Backend : Python / Django REST Framework (DRF)**. Choisi pour sa rapidité de développement, son système de sécurité natif (gestion des utilisateurs) et son ORM puissant.
    - **Modélisation & Versioning :** UML pour les diagrammes de conception, Git/GitHub pour le suivi du code source.
    - **Base de données : PostgreSQL**, système de gestion de base de données relationnelle performant et adapté à Django.

# Chapitre 5. Analyse conceptuelle

## 5.1. Désignation des rôles de l’équipe SCRUM

- **Product Owner :** Responsable de porter la vision fonctionnelle d'**e-tasiko**, de prioriser les fonctionnalités du tableau de bord et de valider les interfaces utilisateurs.
- **Scrum Master :** Garant du respect de la méthodologie Scrum, chargé de lever les obstacles techniques ou organisationnels durant les Sprints.
- **Development Team :** Équipe technique en charge du développement des API de l'application avec Django et de l'intégration de l'interface utilisateur dynamique avec Angular.

## 5.2. Étapes de l’élaboration du product backlog

1. **Collecte et Rédaction :** Traduction des besoins visuels des maquettes en User Stories (US).
2. **Estimation :** Évaluation de la complexité technique de chaque story en Story Points (SP).
3. **Priorisation :** Ordonnancement des stories dans le Backlog en plaçant le cœur du système (Auth et Tâches) en priorité haute.

## 5.3. Product backlog

| ID | User Story | Priorité |
|---|---|---|
| **US1** | En tant qu'utilisateur, je veux me connecter via mon e-mail et mon mot de passe pour accéder de manière sécurisée à mon espace. | Haute |
| **US2** | En tant qu'utilisateur, je veux voir mon tableau de bord contenant mon mini-calendrier, mes catégories d'équipe, mes tâches du jour et mon module de suivi. | Haute |
| **US3** | En tant qu'utilisateur, je veux créer une tâche rapidement via un formulaire modal avec un titre, une date, des étiquettes et un cessionnaire. | Haute |
| **US4** | En tant que collaborateur, je veux lancer et mettre en pause un chronomètre sur une tâche active pour enregistrer mon temps de travail réel. | Moyenne |
| **US5** | En tant qu'équipier, je veux ouvrir le détail d'une tâche pour y ajouter des commentaires et y attacher des pièces jointes (fichiers Excel, Word, etc.). | Moyenne |
| **US6** | En tant qu'utilisateur, je veux accéder à mon profil pour mettre à jour mes informations personnelles et changer ma photo d'avatar. | Moyenne |
| **US7** | En tant qu'utilisateur, je veux filtrer et rechercher mes tâches par projet, date limite, étiquette ou cessionnaire via une barre de recherche avancée. | Basse |

## 5.4. Sprint backlog

### *5.4.1. Présentation du travail à faire (tableau release/sprint)*

- **Sprint 1 : Fondations, Authentification & Profil** (US1, US6) --
  Mise en place de l'API, interface de Login **e-tasiko**, page de gestion du Profil.
- **Sprint 2 : Cœur du Tableau de bord & Gestion des Tâches** (US2, US3) --Vue principale, affichage des listes par période (Aujourd'hui, Demain, Cette semaine), création de tâche.
- **Sprint 3 : Outils de Productivité & Collaboration** (US4, US5, US7)-- Module "Mon suivi" (Chronomètre actif), gestion des pièces jointes, commentaires et filtres de recherche.

### *5.4.2. Planification du sprint 1 (tableau sprint/tâches)*

| **Tâche** | **Estimation (Heures)** | **État** |
|---|---|---|
| Initialisation du projet Django REST API et de l'application Angular | 3h | À faire |
| Modélisation de la base de données PostgreSQL (User, Profile) | 2h | À faire |
| Configuration de l'authentification par Token (JWT) côté Backend | 4h | À faire |
| Création du formulaire d'authentification UI sur Angular (Maquette Organizo) | 4h | À faire |
| Développement de l'écran Profil (Angular) et de l'API d'upload d'avatar (Django) | 5h | À faire |

## 5.5. Dictionnaire des données

- **Utilisateur (User) :** id (Clé primaire), email (Unique, sert d'identifiant), password (Chaîne cryptée), name (Nom complet), profile_picture (Chemin vers l'image de l'avatar).
- **Tâche (Task) :** id (Clé primaire), title (Chaîne de caractères, obligatoire), description (Texte libre), due_date (Date limite), status (Choix : À faire, En cours, Terminée), priority (Choix : Faible, Moyen, Haut), is_archived (Booléen), category_id (Clé étrangère vers Catégorie), assignee_id (Clé étrangère vers Utilisateur).
- **Catégorie (Category) :** id (Clé primaire), name (Nom du groupe/projet).
- **Pièce Jointe (Attachment) :** id (Clé primaire), file (Chemin du fichier sur le serveur), uploaded_at (Date et heure d'envoi), task_id (Clé étrangère).
- **Commentaire (Comment) :** id (Clé primaire), content (Texte), created_at (Date et heure), task_id (Clé étrangère), author_id (Clé étrangère).
- **Suivi Temps (TimeLog) :** id (Clé primaire), start_time (Date/Heure de début), end_time (Date/Heure de fin, optionnel), is_running (Booléen), task_id (Clé étrangère).

## 5.6. Règles de gestion

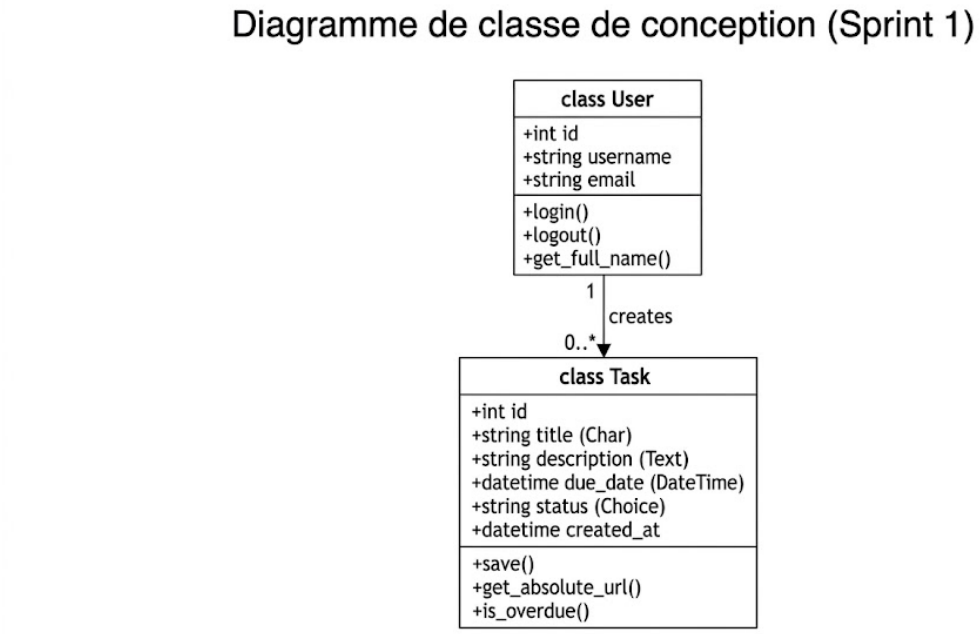
- **RG1 :** Une tâche doit obligatoirement posséder un titre non vide à sa création.
- **RG2 :** Le statut initial attribué par défaut à toute nouvelle tâche créée est "À faire" (ou "Encore comme").
- **RG3 :** Seul un membre associé ou le créateur de la tâche dispose des droits requis pour modifier ses attributs ou la supprimer.
- **RG4 :** Un utilisateur ne peut avoir qu'un seul chronomètre actif (TimeLog.is_running = True) à la fois à un instant T. lancer un nouveau chronomètre met automatiquement le précédent en pause.

## 5.7. Modèle du domaine

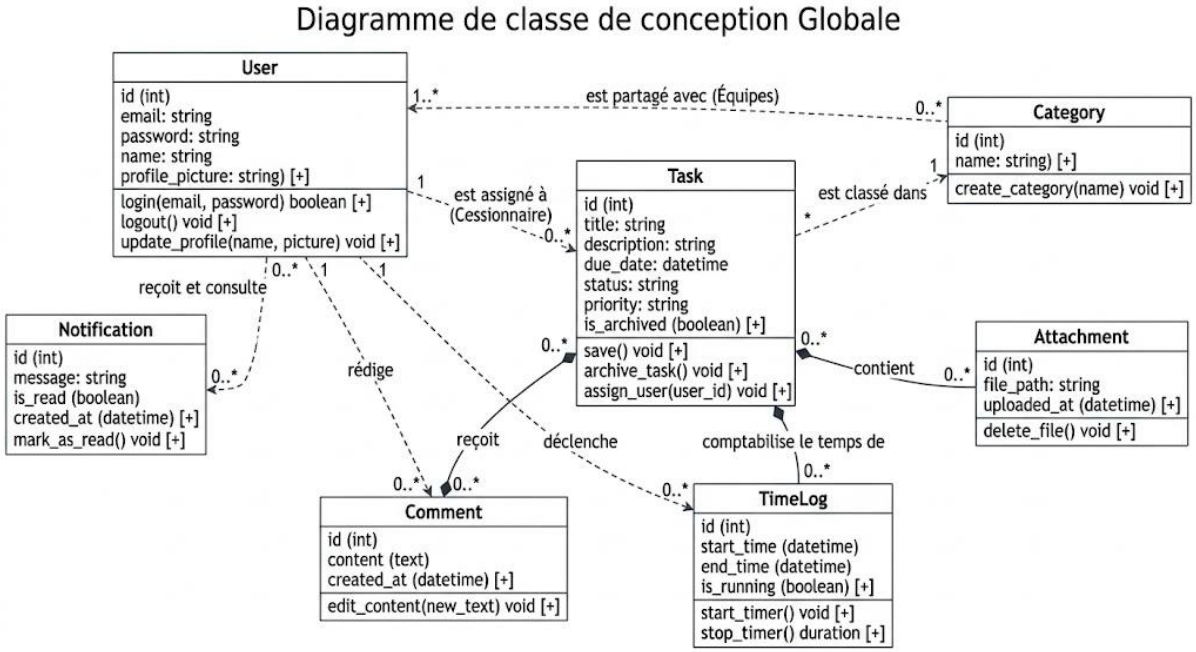
Le domaine d'**e-tasiko** s'articule autour du triptyque fondamental : l'**Utilisateur** (qui agit), la **Tâche** (l'unité de travail qualifiée par des priorités, des statuts, des commentaires et des fichiers joints), et le **Temps** (mesuré précisément via des sessions de tracking).

## 5.8. Diagramme de classe de conception pour le Sprint 1

Ce diagramme restreint le périmètre au strict nécessaire pour démarrer : l'accès sécurisé et la structure fondamentale de la table des tâches.

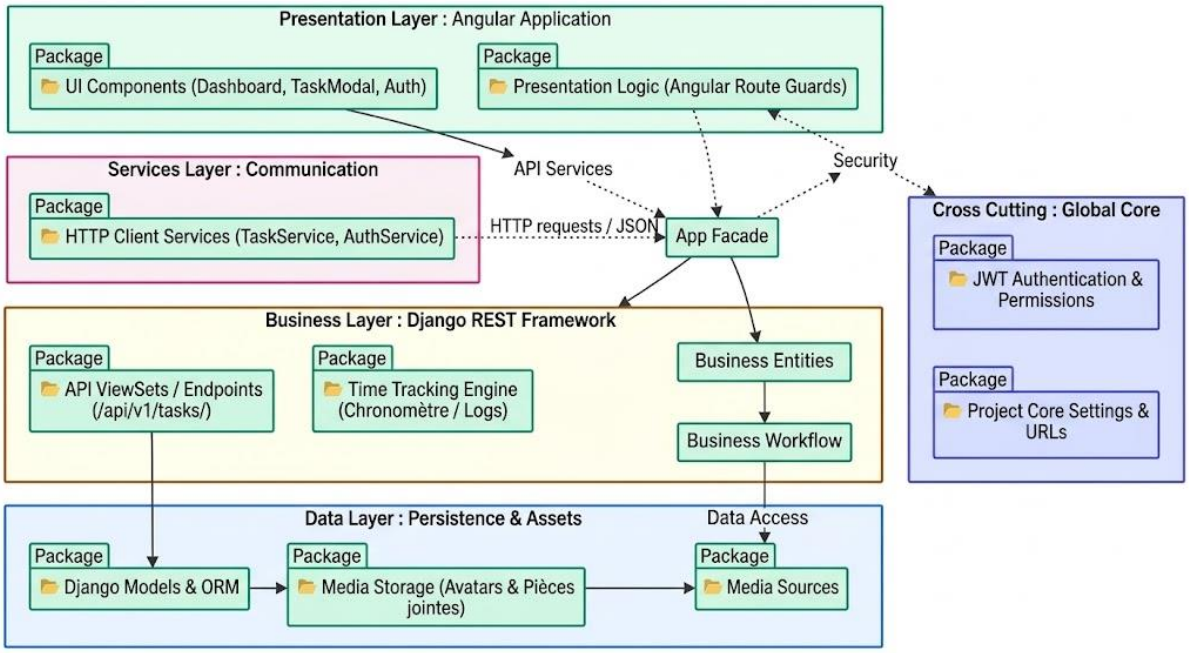


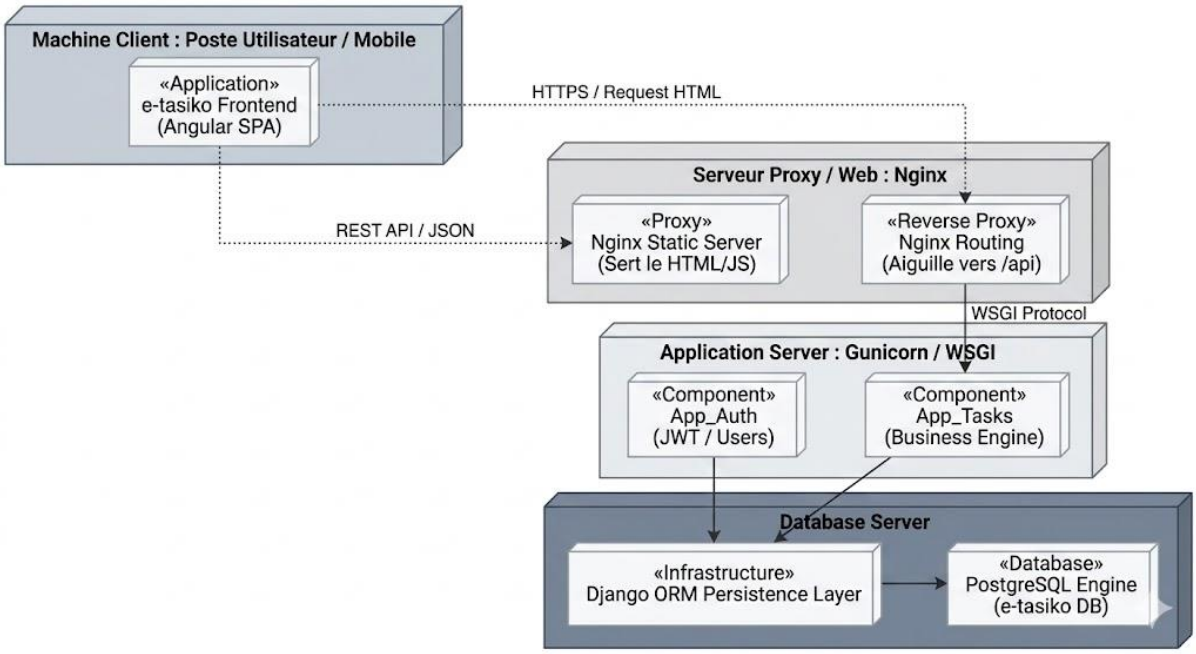

## 5.9. Diagramme de classe de conception globale



## 5.9. Diagramme de paquetage

Il met en évidence l'architecture découplée. L'application Angular communique exclusivement par requêtes HTTP/JSON avec l'API exposée par Django REST Framework.




## 5.10. Diagramme de déploiement